

FUNDAMENTOS DE PROGRAMACIÓN ESTRUCTURADA

Edad actual · Año bisiesto · Estructuras de control

CAMPO	INFORMACIÓN
Evidencia	GA3-220501093-AA2-EV01
Producto	Fundamentos de programación estructurada y estructuras cíclicas
Aprendiz	Jhon Steven Alvarez Ruiz
Programa	Análisis y Desarrollo de Software
Fase	Planeación

De un problema a una solución verificable

Metodología común para los dos algoritmos solicitados.

Esta evidencia aplica las fases de análisis, diseño e implementación a dos problemas autónomos. La solución identifica primero qué información entra y qué resultado debe salir; luego expresa la lógica con estructuras de control y finalmente la representa mediante pseudocódigo y diagramas de flujo.

Objetivos

- Determinar una edad en años cumplidos a partir de dos fechas válidas.
- Clasificar correctamente un año de acuerdo con la regla gregoriana de años bisiestos.
- Usar secuencias, decisiones y ciclos con una finalidad concreta y comprobable.

Tres fases aplicadas

FASE	PREGUNTA	RESULTADO EN ESTE DOCUMENTO
Análisis	¿Qué entra, qué se procesa y qué sale?	Tablas IPO, restricciones y variables.
Diseño	¿En qué orden se toman las decisiones?	Reglas, pasos y diagramas de flujo.
Implementación	¿Cómo se expresa de forma ejecutable?	Pseudocódigo compatible con PSeInt y pruebas.

Simbología de los diagramas

SÍMBOLO	USO
Óvalo	Inicio o fin del algoritmo.
Paralelogramo	Entrada de datos o presentación de resultados.
Rectángulo	Proceso, cálculo o asignación.
Rombo	Decisión con ramas Sí/No.
Flecha	Dirección del flujo y retorno de los ciclos.

Supuesto de calendario: Se usa el calendario gregoriano y el formato YYYY-MM-DD en las pruebas. Para una persona nacida el 29 de febrero, el cumpleaños se considera cumplido el 1 de marzo durante un año no bisiesto.

02 · ALGORITMO 1

Calcular la edad actual

Resultado expresado en años completos, no como una resta aproximada.

Problema: A partir de una fecha de nacimiento y una fecha actual, determinar la edad actual de una persona en años.

2.1 Análisis: entradas, proceso y salida

ENTRADAS	PROCESO	SALIDAS
Día, mes y año de nacimiento. Día, mes y año actuales.	Validar ambas fechas; comprobar el orden; restar años; descontar uno si todavía no ocurrió el cumpleaños.	Edad en años cumplidos o mensaje de datos inválidos.

2.2 Variables y restricciones

VARIABLE	TIPO	REGLA
díaNacimiento / díaActual	Entero	Día existente dentro del mes correspondiente.
mesNacimiento / mesActual	Entero	Valor entre 1 y 12.
añoNacimiento / añoActual	Entero	Valor positivo.
datosVálidos	Lógico	Ambas fechas existen y nacimiento $\leq$ fecha actual.
edad	Entero	Años completos; nunca negativo.
continuar	Carácter	Solo S o N para controlar la repetición.

2.3 Secuencia de solución

1. Solicitar las dos fechas hasta que sean válidas y estén en orden cronológico.
2. Calcular la diferencia entre el año actual y el año de nacimiento.
3. Comparar mes y día para saber si el cumpleaños ya ocurrió en el año actual.
4. Restar uno cuando la fecha actual todavía sea anterior al cumpleaños.
5. Mostrar la edad y permitir que el usuario procese otro caso.

Regla central: $\text{edad} = \text{añoActual} - \text{añoNacimiento}$; si $(\text{mesActual}, \text{díaActual}) < (\text{mesNacimiento}, \text{díaNacimiento})$, entonces $\text{edad} = \text{edad} - 1$.

Diagrama de flujo — calcular la edad

Las flechas de retorno hacen visibles los ciclos de validación y repetición.

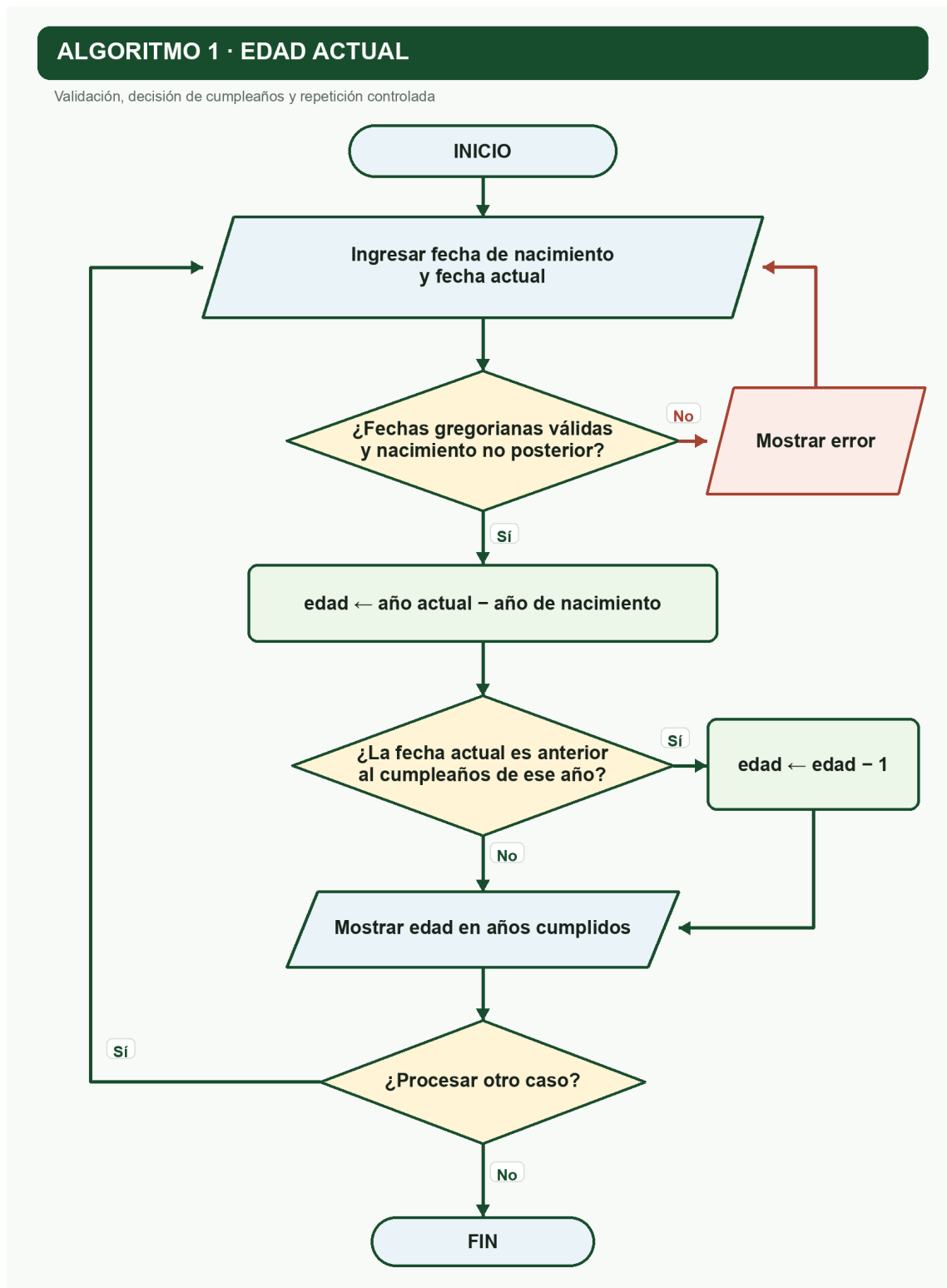


Figura 1. Flujo completo del algoritmo de edad actual.

02 · ALGORITMO 1 · IMPLEMENTACIÓN

Seudocódigo compatible con PSeInt

Incluye validación gregoriana, orden cronológico, cálculo exacto y ciclos controlados.

```
// GA3-220501093-AA2-EV01
// Algoritmo 1: calcular la edad actual en años cumplidos.
SubProceso bisiesto <- EsBisiesto(anio)
    Definir bisiesto Como Logico;
    bisiesto <- (anio MOD 400 = 0) O ((anio MOD 4 = 0) Y (anio MOD 100 <> 0));
FinSubProceso
SubProceso valida <- FechaValida(dia, mes, anio)
    Definir limite Como Entero;
    Definir valida Como Logico;
    valida <- Falso;
    Si anio > 0 Y mes >= 1 Y mes <= 12 Entonces
        Segun mes Hacer
            1, 3, 5, 7, 8, 10, 12:
                limite <- 31;
            4, 6, 9, 11:
                limite <- 30;
            2:
                Si EsBisiesto(anio) Entonces
                    limite <- 29;
                SiNo
                    limite <- 28;
        FinSi
    FinSegun
    valida <- dia >= 1 Y dia <= limite;
FinSi
FinSubProceso
SubProceso ordenValido <- NoEsPosterior(dia1, mes1, anio1, dia2, mes2, anio2)
    Definir ordenValido Como Logico;
    ordenValido <- (anio1 < anio2) O ((anio1 = anio2) Y ((mes1 < mes2) O ((mes1 = mes2) Y (dia1 <= dia2))));
FinSubProceso
Proceso CalcularEdadActual
    Definir diaNacimiento, mesNacimiento, anioNacimiento Como Entero;
    Definir diaActual, mesActual, anioActual, edad Como Entero;
    Definir datosValidos Como Logico;
    Definir continuar Como Caracter;
    continuar <- "S";
    Mientras Mayusculas(continuar) = "S" Hacer
        Repetir
            Escribir "Fecha de nacimiento (dia mes anio):";
            Leer diaNacimiento, mesNacimiento, anioNacimiento;
            Escribir "Fecha actual (dia mes anio):";
            Leer diaActual, mesActual, anioActual;
            datosValidos <- FechaValida(diaNacimiento, mesNacimiento, anioNacimiento) Y FechaValida(diaActual,
mesActual, anioActual);
            Si datosValidos Entonces
                datosValidos <- NoEsPosterior(diaNacimiento, mesNacimiento, anioNacimiento, diaActual, mesActual,
anioActual);
            FinSi
            Si NO datosValidos Entonces
                Escribir "Datos invalidos: revise las fechas y su orden.";
            FinSi
        Hasta Que datosValidos
    edad <- anioActual - anioNacimiento;
    Si (mesActual < mesNacimiento) O ((mesActual = mesNacimiento) Y (diaActual < diaNacimiento)) Entonces
        edad <- edad - 1;
    FinSi
    Escribir "Edad actual: ", edad, " años cumplidos.";
    Repetir
        Escribir "¿Desea procesar otro caso? (S/N):";
        Leer continuar;
        continuar <- Mayusculas(continuar);
    Hasta Que continuar = "S" O continuar = "N"
    FinMientras
FinProceso
```

Pruebas de escritorio

Los casos recorren las ramas antes, durante y después del cumpleaños, además de errores de entrada.

CASO	NACIMIENTO	FECHA ACTUAL	RESULTADO
Cumpleaños pendiente	2000-08-20	2026-08-16	25
Cumpleaños exacto	2000-08-16	2026-08-16	26
Cumpleaños cumplido	2000-08-15	2026-08-16	26
Nacimiento en día bisiesto	2004-02-29	2025-02-28	20
Aniversario tras febrero bisiesto	2004-02-29	2025-03-01	21
Fecha inexistente	2023-02-29	2026-08-16	ERROR
Nacimiento posterior	2027-01-01	2026-08-16	ERROR

Trazabilidad de dos ramas

PASO	CASO PENDIENTE	CASO CUMPLIDO
Diferencia inicial	$2026 - 2000 = 26$	$2026 - 2000 = 26$
Comparación	$08-16 < 08-20 \rightarrow \text{Sí}$	$08-16 < 08-15 \rightarrow \text{No}$
Ajuste	$26 - 1$	Sin ajuste
Salida	25 años	26 años

Resultado de verificación: Las siete pruebas fueron ejecutadas automáticamente por el generador. Las fechas inexistentes o futuras no producen una edad: activan el ciclo de reingreso.

03 · ALGORITMO 2

Determinar si un año es bisiesto

La divisibilidad entre 100 y 400 evita clasificar incorrectamente los años seculares.

Problema: Determinar si un año indicado es o no un año bisiesto.

3.1 Análisis: entradas, proceso y salida

ENTRADA	PROCESO	SALIDA
Año entero positivo.	Validar el año; calcular residuos módulo 4, 100 y 400; aplicar la regla gregoriana.	Mensaje: es bisiesto, no es bisiesto o dato inválido.

3.2 Variables

VARIABLE	TIPO	FUNCIÓN
año	Entero	Dato que se desea clasificar; debe ser mayor que cero.
residuo4 / 100 / 400	Entero	Restos usados para evaluar divisibilidad.
bisiesto	Lógico	Verdadero cuando se cumple la regla gregoriana.
continuar	Carácter	Solo S o N para controlar la repetición.

3.3 Tabla de decisión

CONDICIÓN	CLASIFICACIÓN	EJEMPLO
Divisible por 400	Bisiesto	2000
Divisible por 100, no por 400	No bisiesto	1900
Divisible por 4, no por 100	Bisiesto	2024
No divisible por 4	No bisiesto	2023

Expresión lógica: bisiesto = (año MOD 400 = 0) O ((año MOD 4 = 0) Y (año MOD 100 ≠ 0)).

Diagrama de flujo — año bisiesto

La regla se evalúa después de validar el dato y calcular los tres residuos.

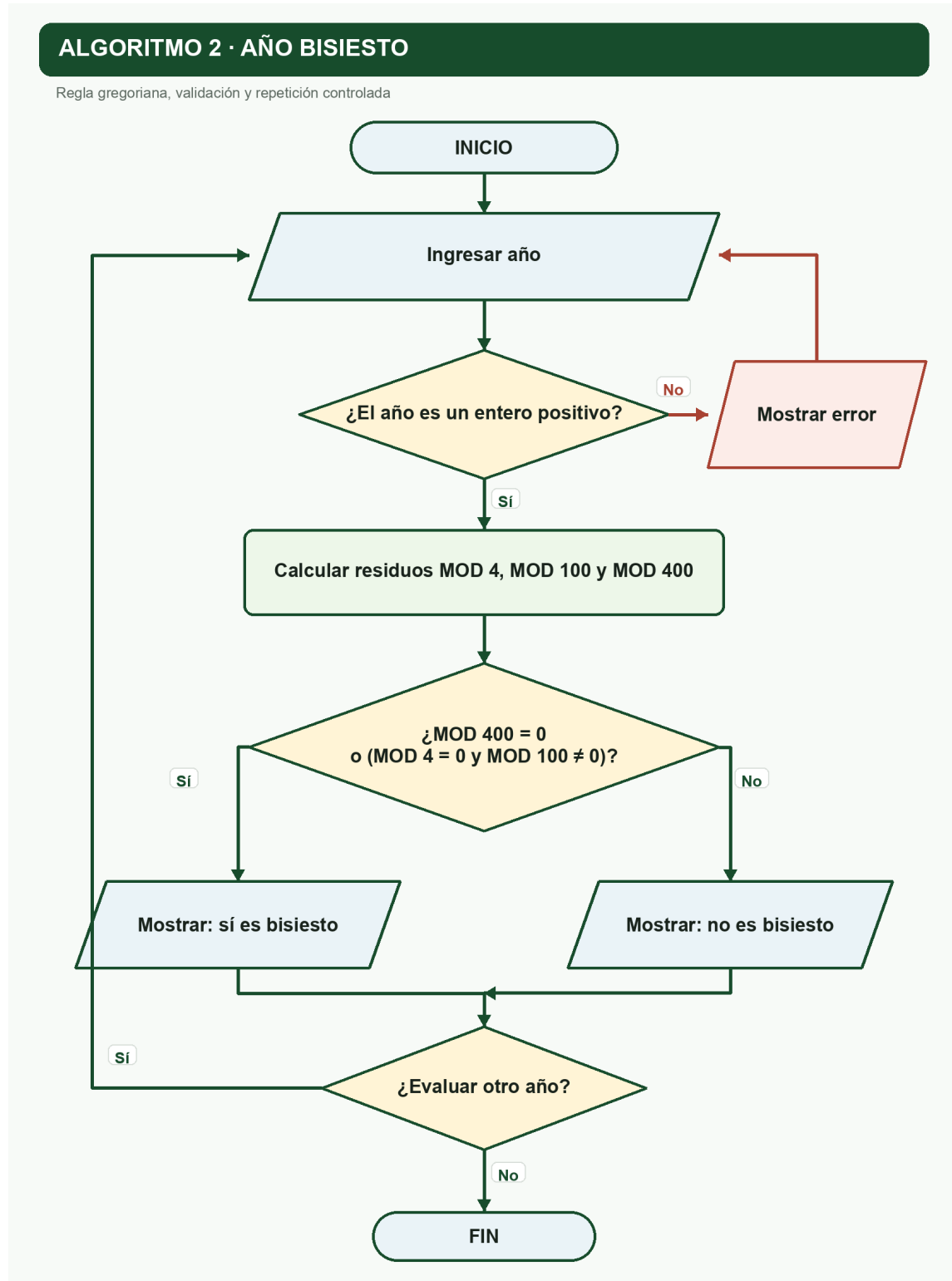


Figura 2. Flujo completo del algoritmo de año bisiesto.

03 · ALGORITMO 2 · IMPLEMENTACIÓN

Seudocódigo y pruebas

La prueba diferencia años ordinarios, múltiplos de cuatro y excepciones seculares.

```
// GA3-220501093-AA2-EV01
// Algoritmo 2: determinar si un año gregoriano es bisiesto.

Proceso DeterminarAñoBisiesto
    Definir año, residuo4, residuo100, residuo400 Como Entero;
    Definir bisiesto Como Logico;
    Definir continuar Como Caracter;

    continuar <- "S";
    Mientras Mayusculas(continuar) = "S" Hacer
        Repetir
            Escribir "Ingrese un año positivo:";
            Leer año;
            Si año <= 0 Entonces
                Escribir "Dato invalido: el año debe ser mayor que cero.";
            FinSi
            Hasta Que año > 0

            residuo4 <- año MOD 4;
            residuo100 <- año MOD 100;
            residuo400 <- año MOD 400;
            bisiesto <- (residuo400 = 0) O ((residuo4 = 0) Y (residuo100 <> 0));

            Si bisiesto Entonces
                Escribir "El año ", año, " es bisiesto.";
            SiNo
                Escribir "El año ", año, " no es bisiesto.";
            FinSi

            Repetir
                Escribir "¿Desea evaluar otro año? (S/N):";
                Leer continuar;
                continuar <- Mayusculas(continuar);
            Hasta Que continuar = "S" O continuar = "N"
        FinMientras
    FinProceso
```

Pruebas de escritorio

AÑO	RESULTADO	JUSTIFICACIÓN
2000	Sí	Divisible por 400
1900	No	Divisible por 100, pero no por 400
2024	Sí	Divisible por 4 y no por 100
2023	No	No es divisible por 4
0	ERROR	El año debe ser positivo

Resultado de verificación: Los cinco casos fueron ejecutados automáticamente. El año cero se rechaza y vuelve a solicitarse mediante el ciclo de validación.

04 · SÍNTESIS

Estructuras usadas y cumplimiento

Correspondencia directa entre la lista de chequeo y la evidencia preparada.

4.1 Estructuras de control

ESTRUCTURA	ALGORITMO DE EDAD	ALGORITMO BISIESTO
Secuencial	Lectura, diferencia de años y salida.	Cálculo de residuos y salida.
Condicional	Validez, orden y cumpleaños pendiente.	Año positivo y regla de divisibilidad.
Cíclica	Reingreso de fechas y procesamiento de otro caso.	Reingreso del año y evaluación de otro caso.

4.2 Correspondencia con la lista de chequeo

INDICADOR	EVIDENCIA PREPARADA
Análisis, diseño e implementación resuelven los problemas.	Dos soluciones completas, coherentes y verificadas con casos normales y límite.
Entradas, salidas y procesos para cada algoritmo.	Tablas IPO, variables, restricciones y secuencias en las páginas 3 y 7.
Solución mediante diagramas de flujo.	Dos diagramas con simbología, ramas Sí/No y ciclos visibles en las páginas 4 y 8.

Conclusiones

- Una edad exacta requiere comparar mes y día; restar únicamente los años puede producir un resultado adelantado.
- La regla completa de año bisiesto debe considerar las excepciones de los múltiplos de 100 y 400.
- Los ciclos de validación evitan continuar con datos imposibles y garantizan una salida coherente.

Fuentes consultadas

- [Guía de aprendizaje GA3 — misma evidencia](#)
- [Material formativo oficial SENA — Análisis de algoritmos](#)
- [National Research Council Canada — Leap years](#)
- [PSeInt — documentación oficial de pseudocódigo](#)
- [ISO 8601 — formato de fechas](#)